

СР3: Предсказание стоимости квартир в Москве

Краткое резюме

Проект решает прикладную ML-задачу: оценить стоимость квартиры в Москве по параметрам объявления и признакам локации. В финальной версии есть воспроизводимый pipeline, сравнение моделей, интерпретация результата и FastAPI-сервис для получения прогнозов.

1. Введение и постановка задачи

Тип задачи — регрессия. Целевая переменная — price, то есть денежная стоимость квартиры. Такой прогноз может быть полезен как быстрый ориентир для сравнения объявлений и проверки адекватности цены относительно площади, этажа, ремонта и расположения.

Основная метрика — RMSLE. Она выбрана потому, что цены недвижимости имеют длинный правый хвост: ошибка в 1 млн рублей по-разному воспринимается для дешёвой и дорогой квартиры. RMSLE делает акцент на относительной ошибке. Дополнительно используются MAE, RMSE и R2: MAE легко интерпретировать в рублях, RMSE сильнее штрафует крупные промахи, R2 показывает общую объяснённую долю дисперсии.

2. Поиск и описание данных

Источник данных — Kaggle Moscow Housing Price Dataset. Датасет выбран, потому что это monetary regression по реальным признакам объявлений о недвижимости. Он не является игрушечной задачей Getting Started, содержит больше 10 000 строк и больше 10 колонок, поэтому подходит для сравнения нескольких моделей.

Показатель	Значение
Строк до очистки	22676
Колонок до очистки	12
Дубликатов удалено	1851
Строк удалено бизнес-правилами	4545
Строк после очистки	16280
Колонок после feature engineering	26
Медианная цена	13500000
99% квантиль цены	400803486

3. Обработка и подготовка данных

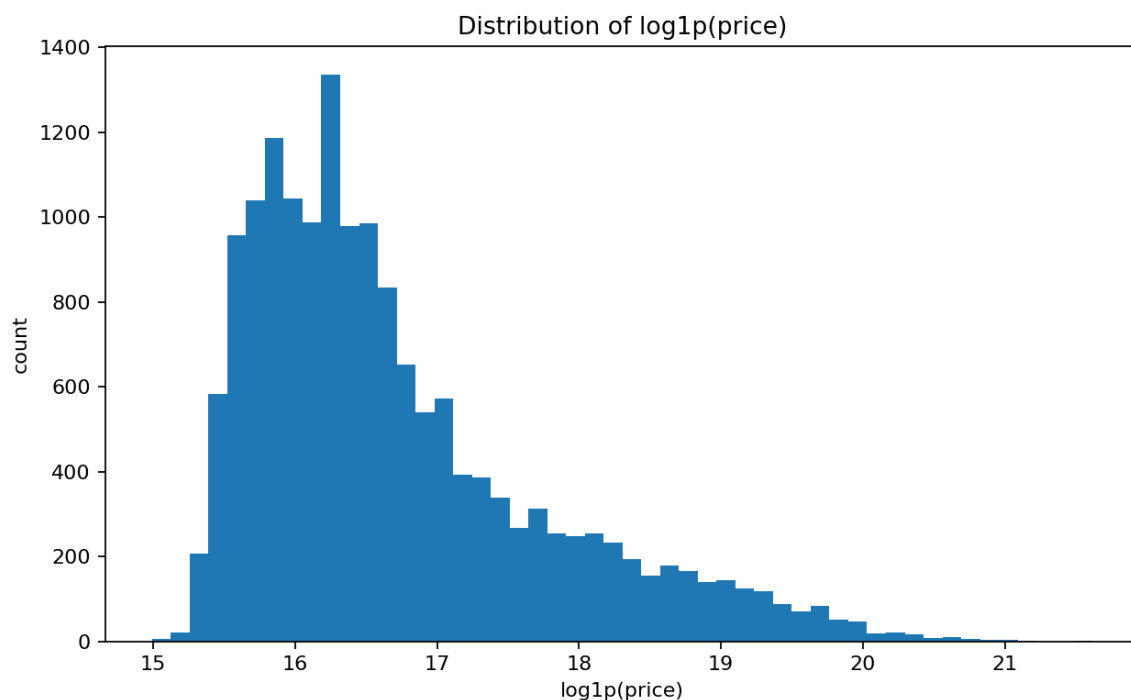
Подготовка данных вынесена в воспроизводимый код. Сначала нормализуются названия колонок и проверяется наличие обязательных признаков. Числовые поля приводятся к числовому типу, категориальные значения очищаются от пустых строк. После этого удаляются полные дубли.

Бизнес-правила удаляют физически невозможные строки: неположительную цену или площадь, нулевую этажность, этаж выше количества этажей в доме, отрицательное расстояние до метро, жилую или кухонную площадь больше общей. Выбросы числовых признаков не считаются на всём датасете: clipping по квантилям находится только на train внутри sklearn Pipeline, что защищает от data leakage.

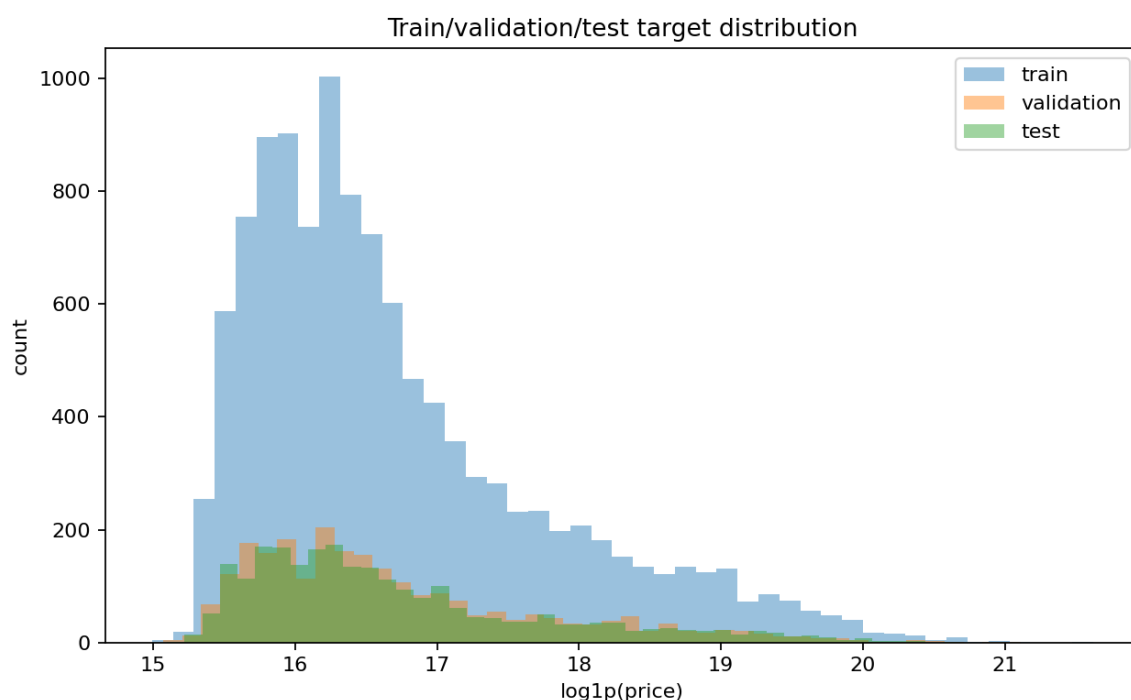
Feature engineering включает признаки площади на комнату, доли жилой и кухонной площади, позицию этажа в доме, признаки первого/последнего этажа, логарифмы площади и расстояния до метро, interaction комнат и площади, отношение кухни к жилой площади, категорию этажа и bucket расстояния до метро.

Разбиение train/validation/test выполнено в пропорции 70/15/15. Для регрессии используется стратификация по ценовым бинам, чтобы распределения цены в частях выборки были похожими.

Распределение $\log_{10}(\text{price})$



Сравнение target на train/validation/test



4. Baseline-модель

В качестве sanity baseline используется `DummyRegressor`, который предсказывает медиану цены. Он нужен, чтобы убедиться, что ML-модели действительно лучше константного прогноза. Второй baseline — `KNeighborsRegressor` без feature engineering: простая модель из коробки, которая даёт начальную точку сравнения для более сложных ансамблей.

5. Эксперименты

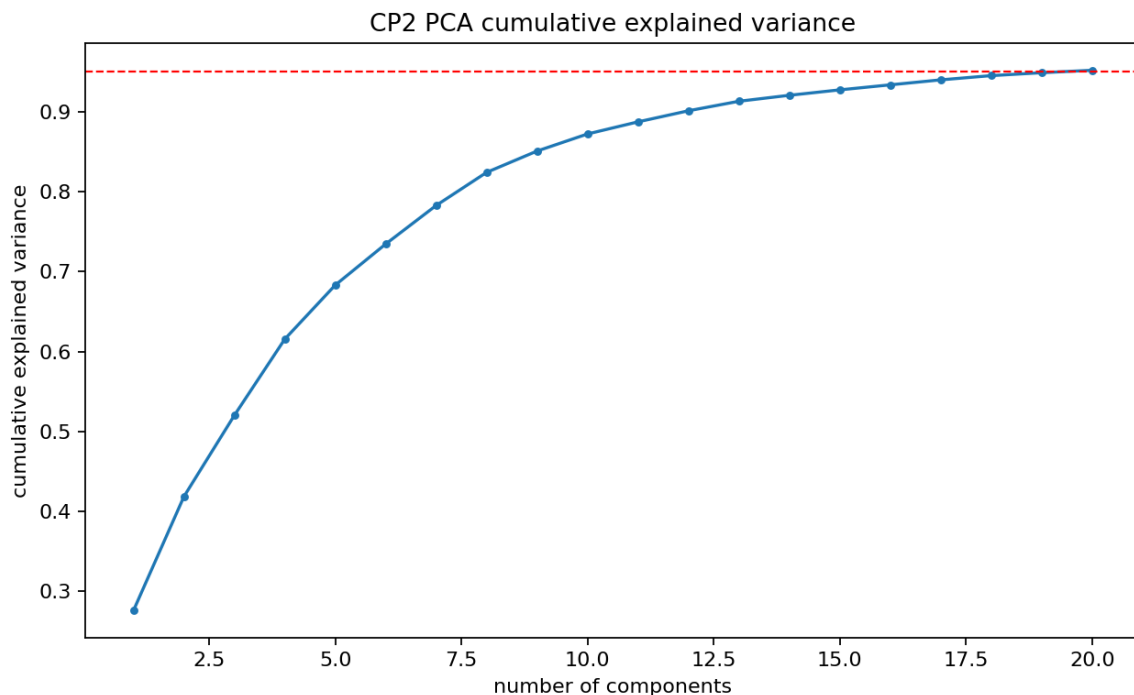
Эксперименты построены вокруг нескольких гипотез: линейная регуляризация может быть сильной на табличных признаках; ансамбли деревьев лучше моделируют нелинейные зависимости цены;

градиентный бустинг должен быть наиболее сильным кандидатом; PCA проверяет, можно ли сжать encoded-признаки без большой потери качества.

model	feature_set	val_rmsle	val_mae	val_r2	fit_time_sec
hist_gradient_lr_007_leaf_31	engineered	0.2105	1.034e+07	0.7364	4.149
hist_gradient_lr_004_leaf_63	engineered	0.2111	1.047e+07	0.7256	9.69
hist_gradient_lr_005_leaf_31	engineered	0.2131	1.057e+07	0.7281	4.308
ridge_alpha_1	engineered	0.2267	1.193e+07	0.6319	0.128
extra_trees_depth_18_leaf_2	engineered	0.2339	1.096e+07	0.726	2.133
extra_trees_depth_none_leaf_3	engineered	0.2367	1.135e+07	0.7141	2.139
random_forest_depth_20_leaf_2	engineered	0.2418	1.172e+07	0.7025	3.769
random_forest_depth_14_leaf_3	engineered	0.2469	1.199e+07	0.6942	3.076
ridge_alpha_30	engineered	0.251	1.302e+07	0.622	0.075
knn_raw_k10	raw	0.2744	1.371e+07	0.6317	0.03
ridge_pca_95	engineered	0.3135	1.569e+07	0.6105	0.09
dummy_median_raw	raw	1.096	3.172e+07	-0.08115	0.034

По validation RMSLE лучшей стала конфигурация HistGradientBoostingRegressor с learning_rate=0.07, max_iter=350, max_leaf_nodes=31 и l2_regularization=0.05. PCA-эксперимент показал, что сжатие до 95% дисперсии ухудшает качество для этой задачи, поэтому финальная модель использует полное пространство признаков.

PCA cumulative explained variance



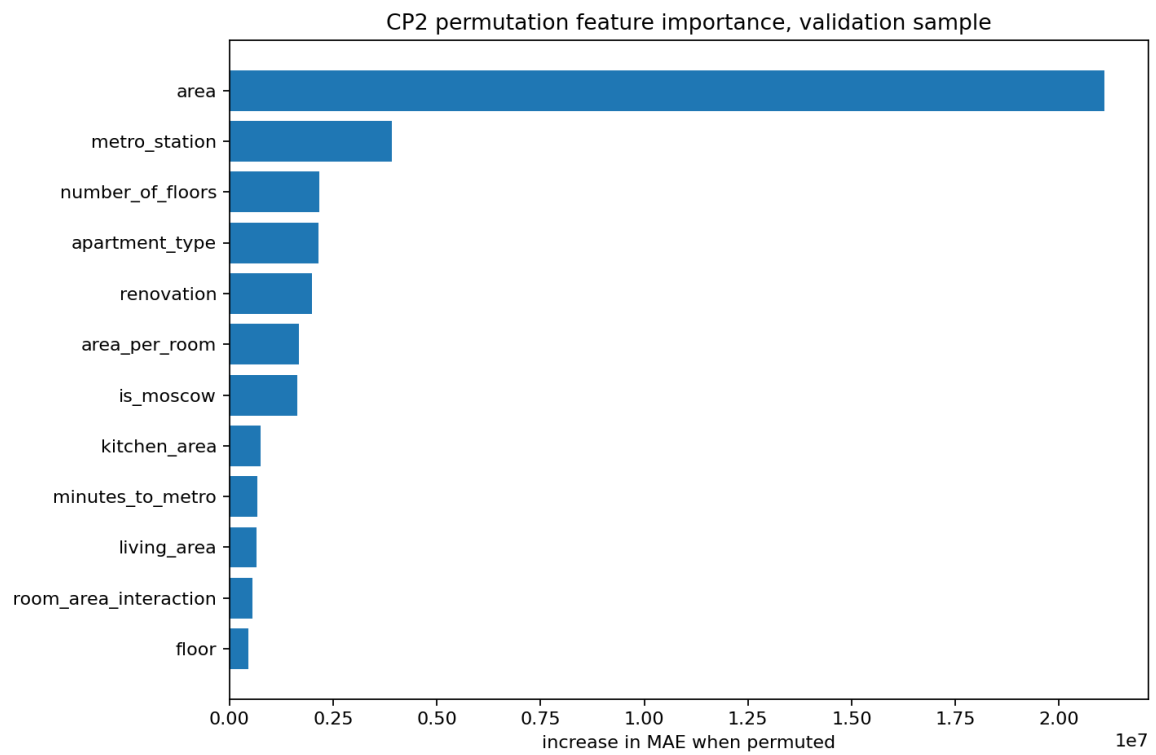
6. Финальная модель и интерпретируемость

Финальная модель: hist_gradient_lr_007_leaf_31. На test она получила RMSLE=0.2135, MAE=9430619.48, RMSE=32346528.12, R2=0.8249.

Для интерпретации используется permutation importance на validation sample. Это не внутренняя важность конкретного алгоритма, а оценка того, насколько растёт ошибка при перемешивании каждого признака. Самыми важными признаками ожидаемо оказались площадь, станция метро, этажность дома, тип квартиры и ремонт.

feature	importance_mean	importance_std
area	2.11e+07	6.971e+05
metro_station	3.913e+06	2.005e+05
number_of_floors	2.172e+06	5.808e+05
apartment_type	2.138e+06	2.067e+05
renovation	1.988e+06	2.432e+05
area_per_room	1.68e+06	1.314e+05
is_moscow	1.633e+06	1.219e+05
kitchen_area	7.424e+05	2.261e+05
minutes_to_metro	6.732e+05	1.053e+05
living_area	6.508e+05	7.828e+04

Permutation feature importance



7. Деплой

Добавлен FastAPI с эндпоинтами GET /health, GET /model-info и POST /predict. Сервис загружает сохранённую финальную модель, валидирует входные признаки через Pydantic и возвращает предсказанную цену в рублях. Swagger UI доступен по адресу <http://127.0.0.1:8000/docs>.

API можно запустить командой `make api`, а быстрый smoke-test без браузера выполняется через `make api-smoke`. Для контейнерного запуска подготовлены `Dockerfile` и `docker-compose.yml`.

API docs

Moscow Housing Price Prediction API 0.3.0 OAS 3.1

[/openapi.json](#)

FastAPI service for predicting Moscow apartment prices from listing features.

default

GET	/health	Health	⌵
GET	/model-info	Model Info	⌵
POST	/predict	Predict	⌵

Schemas

ApartmentFeatures	>	Expand all	object
HTTPValidationError	>	Expand all	object
HealthResponse	>	Expand all	object
ModelInfoResponse	>	Expand all	object
PredictionResponse	>	Expand all	object
ValidationError	>	Expand all	object

Predict request

Moscow Housing Price Prediction API0.3.0OAS 3.1

openapi.json

FastAPI service for predicting Moscow apartment prices from listing features.

default

GET /healthHealth

GET /model-infoModel Info

POST /predictPredict

Parameters

No parameters

Request bodyrequiredapplication/json

Edit ValueSchema

```
{  "apartment_type": "Secondary",  "metro_station": "Kosmopkha",  "minutes_to_metro": 7,  "region": "Moscow",  "number_of_rooms": 2,  "area": 50,  "living_area": 30,  "kitchen_area": 10,  "floor": 5,  "number_of_floors": 10,  "renovation": "cosmetic"}}
```

Execute

Responses

Code	Description	Links
200	Successful Response Media typeapplication/json Controls Accept header. Example ValueSchema <pre>{ "predicted_price": 0, "model_name": "string", "currency": "RUB"}}</pre> <td>No links</td>	No links
422	Validation Error Media typeapplication/json Example ValueSchema <pre>{ "detail": [{ "loc": ["string", 0], "msg": "string", "type": "string" }]}</pre>	No links

Schemas

ApartmentFeaturesExpand allobject

HTTPValidationErrorExpand allobject

HealthResponseExpand allobject

ModelInfoResponseExpand allobject

PredictionResponseExpand allobject

ValidationErrorExpand allobject

Predict response

Moscow Housing Price Prediction API 0.3.0 OAS 3.1
Swagger JSON

FastAPI service for predicting Moscow apartment prices from listing features.

default

GET /health Health

GET /model-info Model Info

POST /predict Predict

Parameters

No parameters

Request body required

application/json

Edit Value Schema

```
{  "apartment_type": "Secondary",  "metro_station": "Kosyakovskaya",  "distance_to_metro": 7,  "region": "Moscow",  "number_of_rooms": 2,  "area": 58,  "living_area": 38,  "kitchen_area": 18,  "floor": 5,  "number_of_floors": 18,  "renovation": "cosmetic"}
```

Execute Clear

Responses

Curl

```
curl -X 'POST' \  https://127.0.0.1:8080/predict/ \  -H 'accept: application/json' \  -H 'content-type: application/json' \  -d '{  "apartment_type": "Secondary",  "metro_station": "Kosyakovskaya",  "distance_to_metro": 7,  "region": "Moscow",  "number_of_rooms": 2,  "area": 58,  "living_area": 38,  "kitchen_area": 18,  "floor": 5,  "number_of_floors": 18,  "renovation": "cosmetic"  }'
```

Request URL

http://127.0.0.1:8080/predict

Server response

Code Details

200

Response body

```
{  "predicted_price": 1648677.1541000001,  "model_name": "task_gradient_boost_1r_200_1_def_31",  "currency": "RUB"  }
```

Response headers

```
content-length: 99  content-type: application/json  date: Sat, 15 Mar 2025 13:18:33 GMT  server: uvicorn
```

Responses

Code	Description	Links
200	Successful Response	No links
422	Validation Error	No links

Media type

application/json

Content Accept header

Example Value Schema

```
{  "predicted_price": 0,  "model_name": "string",  "currency": "RUB"  }
```

Media type

application/json

Example Value Schema

```
{  "detail": {    "loc": [      "string",    ],    "msg": "string",    "type": "string"  }  }
```

Schemas

- ApartmentFeatures > Expand all object
- HTTPValidationError > Expand all object
- HealthResponse > Expand all object
- ModelInfoResponse > Expand all object
- PredictionResponse > Expand all object
- ValidationError > Expand all object

Видео демонстрации

Видео работы API: <https://drive.google.com/file/d/1WevJNReWjvA7DrICg0kZgCtoC6JF3y3M/view?usp=sharing>

8. Заключение и выводы

Проект доведён до полного ML-пайплайна: данные очищаются и обогащаются признаками, модели сравниваются по единой validation-схеме, финальная модель проверяется на test и доступна через API. Основное ограничение — модель обучена на одном Kaggle-датасете, поэтому при переносе на реальные новые объявления полезно периодически переобучать её и мониторить drift признаков и цен.